

Agentic Design Space Exploration for Joint Hardware Configuration Selection and Mapping of AI Inference Workloads on Heterogeneous Edge SoCs

Geetha Prasuna Yarramneni[§], Surya Selvam[§], Wilfried Haensch^{##} and Anand Raghunathan[§]

[§]Elmore Family School of ECE, Purdue University, West Lafayette, IN, USA

^{##}Pritzker School of Molecular Engineering, University of Chicago, Chicago, IL, USA

^{*}Argonne National Laboratory, Argonne, IL, USA

USA

Abstract

Modern edge Systems-on-Chip (SoCs) integrate heterogeneous processing units such as CPUs, GPUs, and NPUs, each with distinct performance and energy characteristics. Deploying AI inference workloads on such platforms under real-time latency and energy constraints requires jointly mapping workloads to processing units (PUs) and configuring each PU (e.g., by selecting the number of active cores and operating frequency). This joint space grows combinatorially, making exhaustive search for the optimal deployment strategy computationally infeasible. Most prior work on design space exploration (DSE) has applied black-box optimization (BBO) methods such as evolutionary search to navigate such vast design spaces. An evaluation (simulation or execution on real hardware) is used to obtain metrics, such as latency and energy, to guide the search. Recently, Large Language Model (LLM)-guided approaches for DSE have gained interest but continue to use sparse feedback similar to BBO approaches. We observe that sparse feedback limits the efficiency of LLM-driven DSE since (i) it does not foster insight into the nature of the design space or the specific reasons that lead a given design choice performing the way it does, and (ii) it does not leverage the full reasoning abilities of LLMs, which are capable of taking richer information from the evaluation of each design point to make higher-quality decisions.

Based on these insights, we present TraceDSE, an agentic DSE flow that performs joint workload mapping and PU configuration selection for AI inference workloads on heterogeneous SoCs. TraceDSE consists of an iterative proposer–critic loop driven by rich feedback in the form of system execution traces. The LLM proposer agent generates candidate mappings and PU configurations for hardware evaluation. The LLM critic agent, equipped with programmatic trace-analysis tools, analyzes execution traces to identify bottlenecks and suggest targeted refinements. This feedback loop enables deeper insights into each design point, higher-quality decisions, and eventually a more effective search of the design space.

Across four AI inference workloads (models of varying complexity and an application pipeline composed of multiple models) deployed on an Intel Meteor Lake SoC, TraceDSE consistently outperforms two state-of-the-art BBO tools. Specifically, TraceDSE improves Pareto frontier hypervolume by up to 35% over NSGA-II evolutionary

search and by up to 68% over Bayesian optimization, while requiring ~6–9× fewer hardware evaluations.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**; • **Hardware** → **Electronic design automation**.

Keywords

Design space exploration, Heterogeneous SoC, AI inference, LLM agents, Multi-objective optimization

ACM Reference Format:

Geetha Prasuna Yarramneni[§], Surya Selvam[§], Wilfried Haensch^{##} and Anand Raghunathan[§]. 2026. Agentic Design Space Exploration for Joint Hardware Configuration Selection and Mapping of AI Inference Workloads on Heterogeneous Edge SoCs. In *2026 ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD '26)*, September 07–09, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3831599.3840344>

1 Introduction

AI models are being widely deployed on edge hardware such as mobile phones, wearables, smart glasses, and AR/VR headsets, powering on-device applications across vision (face recognition, gesture and gaze tracking, and computational photography), speech (transcription and translation), and natural language processing (personal assistants) [3]. These devices typically contain heterogeneous SoCs that integrate multiple PUs—CPUs, GPUs, and NPUs. Efficiently deploying models onto such SoCs requires jointly navigating two design spaces: the *workload mapping space* and the *PU configuration space*. **(1) Workload mapping space:** A neural network workload is commonly represented as a computational graph, where each node corresponds to a computational kernel, such as a convolution, fully-connected layer or attention computation. Mapping such a graph onto a heterogeneous SoC requires selecting which PU each node executes on. Each PU could offer a different latency–energy trade-off for the same node. For example, a node that upsamples a 48×48 feature map to 192×192 and applies a 1×1 convolution ($192 \rightarrow 1000$ channels, ~7 GFLOPs) when executed on an Intel Meteor Lake SoC takes 46.7 ms and 1.40 J on the GPU, 51.3 ms and 1.04 J on the NPU, and 51.5 ms and 3.81 J on the CPU—the GPU is 10% faster but costs 26% more energy than the NPU, while the CPU matches the NPU’s latency at 3.7× the energy cost. Beyond these node-level tradeoffs, the overall goal is to utilize the collection of PUs to achieve the desired overall latency and energy goals for the full application. **(2) PU configuration space:** Each PU exposes runtime-configurable



This work is licensed under a Creative Commons Attribution 4.0 International License. *MLCAD '26, Jeju Island, Republic of Korea*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2878-5/2026/09

<https://doi.org/10.1145/3831599.3840344>

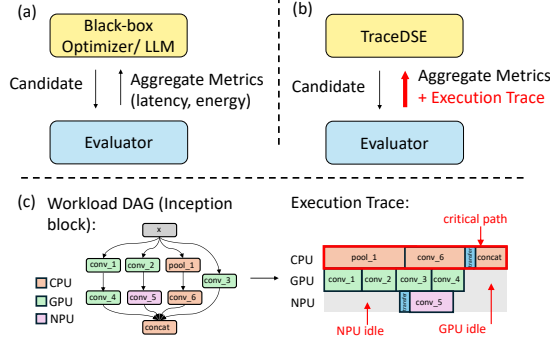


Figure 1: (a) Conventional DSE: the optimizer receives only aggregate metrics. (b) TraceDSE: the evaluator also returns an execution trace, enabling targeted refinement. (c) Execution trace of a block of the Inception DNN revealing opportunities that aggregate metrics miss.

options that further shape performance–energy behavior. For example, on an Intel Meteor Lake SoC, the CPU offers a choice between Performance cores (P-cores) and Efficiency cores (E-cores) with a configurable active core count. The GPU and NPU expose operating frequency control, and the NPU additionally exposes a tile count (number of active compute tiles). The cross-product of choices in these two spaces yields a vast and complex design space. For example, deploying HRNet [18], a pose-estimation model with 98 nodes, across 3 PUs on an Intel Meteor Lake SoC—each PU offering 2 configurations—yields a design space with $3^{98} \times 2^3 \approx 4.6 \times 10^{47}$ candidates. Each candidate requires 30–60 seconds per hardware evaluation, making exhaustive search infeasible.

Navigating this design space to identify candidates that achieve the best latency–energy trade-offs requires an efficient search strategy. Prior iterative DSE methods follow a common paradigm (Figure 1(a)): an optimizer proposes a candidate (or a population of candidates), an evaluator returns aggregate metrics (e.g., end-to-end latency and energy), and the optimizer uses this feedback to propose the next candidate(s). Black-box optimizers (BBOs) such as genetic algorithms and Bayesian optimization operate within this loop, relying solely on aggregate metrics to guide the search [10, 17]. These methods lack the domain knowledge to avoid inferior regions of the design space, leading to slow convergence. Prior work has addressed this by embedding hand-crafted domain-specific heuristics into these optimizers [2, 4, 17], but such heuristics require manual engineering effort for each new platform or workload. More recently, LLM-guided approaches have automated this step, leveraging knowledge embedded in pre-trained LLMs to guide the search [6, 11–14, 20]. All these methods still rely on aggregate metrics, limiting search efficiency.

Evaluating a candidate on hardware has the potential to yield far more insight than aggregate metrics like latency and energy. An execution trace can provide information about the execution time and energy of each node, cross-PU data transfer times, PU utilization, and more. Figure 1(c) illustrates this on a block from the Inception DNN. The execution trace reveals that the critical-path nodes are mapped to the CPU—making it the bottleneck—and that nodes conv_2 and conv_3 are queued on the GPU while the NPU remains idle. These insights directly suggest two design choices:

remapping queued nodes onto idle PUs to shorten the critical path and increasing the active CPU core count to accelerate the critical-path nodes. These decisions cannot be easily derived from aggregate metrics alone.

Deriving actionable insights from a raw execution trace requires analyzing per-node execution behavior and reasoning about which mapping and configuration changes would improve performance. LLMs, with their ability to interpret structured representations—execution traces for performance analysis, or workload graphs for mapping decisions—are well-suited to automate this within the DSE loop. These two reasoning tasks are distinct: execution analysis demands detailed per-node traces that grow across iterations, while candidate generation requires a macro view of the design space. This motivates decomposing the DSE loop into two specialized agents: one that analyzes execution traces and recommends improvements, and one that generates new candidates.

The key contributions of this work are as follows:

- We present TraceDSE, an agentic DSE flow for joint mapping and PU configuration selection.
- We propose an iterative proposer–critic loop in which the proposer generates candidates and the critic analyzes their execution traces and recommends targeted refinements to the proposer. To balance refinement with exploration, a search state maintains the Pareto front from the design points explored thus far and exposes under-evaluated regions of the design space to the proposer.
- We equip the critic with programmatic trace-analysis tools that provide insight into how individual mapping and configuration choices affect latency and energy, enabling the proposer to act on richer, structured feedback rather than on aggregate metrics alone.
- We evaluate TraceDSE on four AI inference workloads (models of varying complexity and an application pipeline composed of multiple models) executed on an Intel Meteor Lake SoC. TraceDSE consistently outperforms two state-of-the-art BBO tools. Specifically, TraceDSE improves Pareto frontier hypervolume by up to 35% over NSGA-II evolutionary search and by up to 68% over Bayesian optimization, while requiring ~ 6 – $9\times$ fewer hardware evaluations.

2 Related Work

Existing DSE methods employ different search strategies within an optimizer–evaluator loop. ArchGym [10] provides a unified framework for comparing search algorithms, including evolutionary algorithms, Bayesian optimization, reinforcement learning, and ant colony optimization, across architectural design spaces. MOHaM [4] uses a multi-objective evolutionary algorithm to jointly search chiplet configurations, chiplet placement, and layer-to-chiplet assignments for multi-DNN deployment. FARSI [2] augments simulated annealing with architectural heuristics that leverage system-level estimates of compute and memory bottlenecks to guide exploration within the SoC design space. Spotlight [17] uses Bayesian optimization for accelerator HW/SW co-design by replacing raw design parameters with derived features, such as PE utilization and estimated DRAM traffic, in the surrogate model.

Recent work integrates LLMs into the iterative DSE loop. gem5 Co-Pilot [6] uses an LLM to retrieve and reason over prior simulation results to guide L2 cache design. LEMOE [11] searches RISC-V

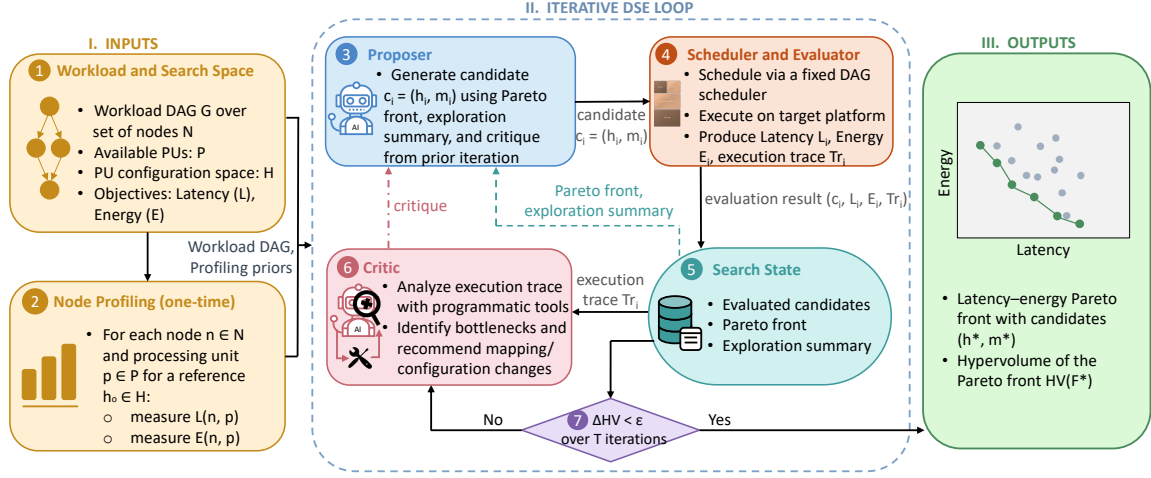


Figure 2: TraceDSE framework overview. The proposer generates candidates using the workload DAG, profiling priors, and prior-iteration feedback. The critic analyzes execution traces with programmatic tools to recommend changes.

microarchitecture parameters using multi-objective Bayesian optimization, where an LLM warm-starts the search and enhances the surrogate model through in-context learning. iDSE [12] uses LLM-driven convergent and divergent reasoning to refine HLS directive configurations towards the Pareto front. LLM-DSE [20] coordinates multiple LLM agents to iteratively propose and refine HLS directives. LLAMBO [13] applies LLMs to Bayesian optimization for ML hyperparameter tuning by expressing its core components—warm-starting, candidate sampling, and surrogate modeling—as natural-language prompts. LLM-Box [14] uses an LLM to warm-start the search and filter infeasible candidates before handing off to standard multi-objective BBO for mapping LLM inference workloads onto AI accelerator pods. These methods leverage the prior knowledge embedded in pretrained LLMs to improve search efficiency compared with conventional optimizers; however, the feedback they consume is limited to aggregate metrics. We argue that such an approach does not lend to attributing outcomes to individual design decisions, nor does it provide a basis for reasoning about how to improve the current solution.

3 TraceDSE Methodology

In this section, we formulate the joint mapping and configuration problem (Section 3.1), present the TraceDSE framework (Section 3.2), and describe each component in detail (Sections 3.3-3.6).

3.1 Problem Formulation

We represent the workload as a directed acyclic graph (DAG) G , with a node set N and edges that encode data dependencies. The target platform provides a set of heterogeneous PUs, $P = \{p_1, p_2, \dots, p_{|P|}\}$, with full PU configuration space $H = H_1 \times \dots \times H_{|P|}$, where H_i denotes PU-specific parameters for PU p_i (e.g., core type and active core count for CPUs, tile allocation for NPU). A mapping $m : N \rightarrow P$ assigns each node to a PU. Given a mapping m and PU configuration $h \in H$, a scheduler produces a schedule $S(G, h, m)$ that respects the data dependencies in G . We optimize two objectives: the end-to-end execution time (latency) $L(h, m) = \text{Makespan}(S(G, h, m))$, and the energy $E(h, m) = \text{Pow}(h, m) \cdot L(h, m)$, where $\text{Pow}(h, m)$ is the average system power during inference. The quality of a Pareto front $\mathcal{F}$ is measured by the hypervolume $\text{HV}(\mathcal{F})$ [23], which

denotes the area in the objective space dominated by $\mathcal{F}$. The goal is to discover nondominated configurations that maximize $\text{HV}(\mathcal{F})$ in the latency–energy space.

3.2 Framework Overview

TraceDSE is a closed-loop DSE framework consisting of two LLM agents—a proposer and a critic—connected through an iterative evaluate-analyze-refine loop (Figure 2). Before the loop begins, one-time node profiling measures per-node latency and energy on each PU. In each iteration, the proposer generates candidate design points, which are scheduled and evaluated on the target platform, producing aggregate metrics and an execution trace. The critic analyzes this trace using programmatic tools (Table 1) and recommends targeted mapping and configuration changes to the proposer. A search state records evaluated candidates and maintains the Pareto front and exploration summary, which the proposer uses alongside the critic’s feedback to generate candidates in the next iteration. The loop terminates when the relative improvement in Pareto-front hypervolume remains below a specified threshold (0.5% in our experiments) over a specified number of consecutive iterations (five in our experiments).

3.3 Node Profiling

Before the DSE loop begins, node profiling produces per-node latency and energy measurements on each PU, which we refer to as *profiling priors*. TraceDSE obtains these by running the workload on the target platform once per PU, mapping all nodes to that PU under maximum resource settings. The profiling priors serve two roles: they warm-start the proposer for generating informed initial candidates, and they enable the critic to estimate the end-to-end latency impact of a proposed remapping by substituting a node’s profiled latency on the proposed PU without additional hardware evaluation.

3.4 Proposer Agent

The proposer generates a batch of candidate design points in each iteration. Each candidate consists of a mapping, a PU configuration, and a reasoning statement explaining its design choices.

To generate these candidates, the proposer must determine a mapping of nodes to PUs and a configuration for each PU. We

provide four inputs that inform these decisions: the *workload DAG*, *PU configuration space*, *optimization objectives*, and *profiling priors* (Section 3.3). The workload DAG lets the proposer exploit concurrency by distributing independent nodes across PUs while respecting ordering constraints. The profiling priors let the proposer avoid clearly inefficient PU choices when constructing initial mappings.

In subsequent iterations, the proposer receives three additional inputs to guide its search. The *Pareto front* captures the best latency–energy trade-offs discovered at any point in the search, guiding the proposer toward regions in the objective space where new candidates would most improve hypervolume. The *exploration summary* records which mapping and configuration regions have been evaluated, so the proposer avoids revisiting them. The *critique* from the prior iteration provides per-candidate feedback identifying which design choices drove improvements in the metrics, along with recommended changes to the mapping and PU configuration, which the proposer uses to inform its next batch of candidates.

3.5 Scheduler & Evaluator

Given a candidate design point, the scheduler assigns nodes to PUs according to the candidate’s mapping and schedules them using a DAG scheduler, Heterogeneous Earliest Finish Time (HEFT) [19], respecting data dependencies in the workload DAG. The evaluator executes the scheduled workload on the target platform and produces the end-to-end latency, energy, per-PU energy breakdown, and an execution trace—a per-node record of the assigned PU, start and end timestamps, compute time, and inter-PU data transfer delays. All evaluation results are stored in the search state.

3.6 Critic Agent

The critic analyzes each candidate’s execution trace and produces a critique consisting of three parts: *per-candidate feedback* identifying which design choices drove the measured metrics, so the proposer knows what to preserve and what to change; *observations* synthesized across current and prior iterations that capture recurring patterns—for example, that a particular node-to-PU mapping consistently underperforms; and *recommended changes* to each candidate’s mapping and configuration, informing the proposer’s next batch of candidates.

To distill the execution trace into actionable feedback, the critic uses six programmatic tools (Table 1) that it autonomously selects and invokes—for example, first extracting the critical path to identify bottleneck nodes, then inspecting those nodes to determine which alternative PU would reduce their latency. Using the profiling priors (Section 3.3), the critic can also estimate the latency impact of a proposed remapping, screening out changes unlikely to improve performance—though these estimates do not capture runtime contention that arises only during hardware execution.

4 Experimental Setup

We evaluated TraceDSE on an Intel Meteor Lake edge SoC comprising a CPU (6 P-cores, 8 E-cores, 2 low-power E-cores), an integrated GPU, and a 2-tile NPU. We use OpenVINO [9] as the inference runtime and Gemini 3.1 Pro Preview [8] (temperature 1.0, thinking level *high*) as the LLM for both the proposer and the critic agents.

Each node in the execution trace is mapped to one of three PUs: CPU, GPU, or NPU. The PU configuration space enumerates valid combinations of five parameters: CPU cores $\in \{1, \dots, 16\}$, CPU core

Table 1: Critic tool interface. The critic autonomously selects tools to analyze an execution trace.

Tool	Returns
Critical-path extraction	Nodes on the critical path with each node’s latency and share of the total latency
Latency bottleneck ranking	Top- k nodes ranked by descending latency with estimated speedup on alternative PUs
PU utilization and idle-interval analysis	Per-PU utilization and idle intervals
Node inspection	Detailed breakdown of a single node’s execution, including compute time, scheduling and communication delays, and estimated latency of the node on each PU
Energy breakdown	Energy consumed by each PU as a percentage of total system energy
Latency estimation	Predicted latency after a proposed mapping or configuration change

type $\in \{\text{P-core, E-core}\}$, hyperthreading $\in \{\text{on, off}\}$, NPU turbo $\in \{\text{on, off}\}$, and NPU tile count $\in \{1, 2\}$.

We evaluate four workloads with varying degrees of parallelism: Depth Anything V2 [22] (28 nodes, mostly sequential), HRNet [18] (98 nodes, repeated fork–join parallelism), ImageBind [7] (72 nodes, four parallel encoder chains fanning into a joint embedding), and a speech-based Visual Question Answering (VQA) pipeline combining Whisper-Small [16], CLIP ViT-L/14 [15], and Qwen2.5-3B [21] (71 nodes, parallel audio and vision encoders fanning into an LLM decoder).

We report the median latency across 50 inference runs after 5 warm-up runs. Energy per inference is computed as $(P_{\text{avg}} - P_{\text{idle}}) \times \text{latency}$, where P_{avg} and P_{idle} are sampled via HWiNFO during inference and when the system is idle, respectively. P_{avg} is polled from HWiNFO’s shared-memory sensors every 0.3 s (250 ms hardware refresh) across each configuration’s active window of 50 repeated inferences (8–30 s, 27–94 samples), so sub-second per-inference latency is amortized over tens of samples rather than read from a single sample. Across re-measurements of all the evaluated candidates, the coefficient of variation is 0.3–10.8% for energy (median 2.8%) and 0.16–1.2% for latency.

We compare TraceDSE against three baselines: random search, Non-dominated Sorting Genetic Algorithm II (NSGA-II) [5] (population size 16, uniform crossover), and multi-objective Bayesian optimization using Tree-structured Parzen Estimators (TPE) [1], which natively supports the mixed continuous-categorical parameter space. Each baseline is run for a fixed 12-hour wall-clock budget on the target SoC. All methods start from the same three seed candidates (all nodes on a single PU, maximum resource settings), counted toward each method’s budget. We normalize objectives per benchmark—using the maximum latency and maximum energy across all methods’ Pareto-optimal candidates as the reference point—and report $\widehat{HV} \in [0, 1]$.

5 Results

We next present results of our experiments evaluating TraceDSE.

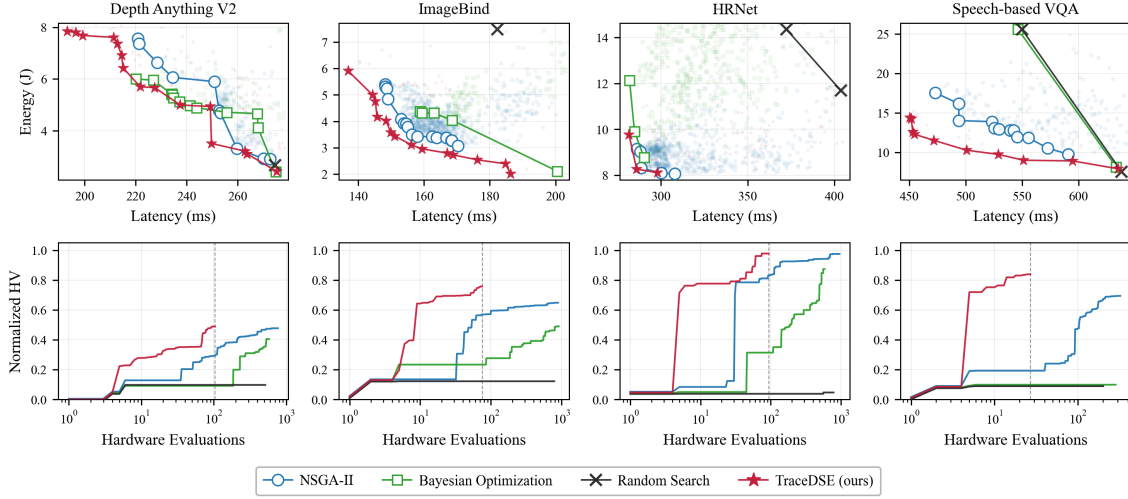


Figure 3: Pareto fronts (top) and normalized hypervolume vs. hardware evaluations (bottom, log scale). Faded points are dominated. The dashed line marks TraceDSE’s termination; baselines have completed only 11–18% of their evaluations.

5.1 Pareto Front Quality and Evaluation Efficiency

TraceDSE achieves the highest hypervolume on all four benchmarks while requiring 5.6–9.0 $\times$ fewer evaluations (Table 2). At iso-latency, TraceDSE achieves 20–32% lower energy than the strongest baseline on ImageBind and Speech-based VQA (Figure 3, top). At iso-energy, TraceDSE achieves 7–18% lower latency on ImageBind, HRNet, and Speech-based VQA. On three of four benchmarks, TraceDSE discovers low-latency Pareto-front extremes unreachable by any baseline—extending 7–14% beyond the fastest baseline—by exploiting the critic’s visibility into idle PU intervals and critical-path bottlenecks.

TraceDSE’s advantage is larger when the workload offers more opportunities for concurrent execution across PUs. On HRNet, which has repetitive fork–join parallelism, TraceDSE is only marginally better than NSGA-II (1.1% hypervolume gap). On ImageBind and Speech-based VQA, where independent encoder chains can be distributed across PUs, the gap widens to 25–29%. The convergence plots (Figure 3, bottom) further illustrate the sample efficiency of trace-driven feedback: TraceDSE reaches near-final hypervolume within 10–30 evaluations and terminates with a higher hypervolume than any baseline achieves after its full 12-hour budget. Of TraceDSE’s total wall-clock time, hardware evaluation accounts for 60–65%, critic LLM inference for 25–35%, and proposer LLM inference for 4–13%; evaluation on the target SoC remains the dominant cost.

5.2 Ablation Study

We ablate four design choices of TraceDSE across all four benchmarks (Table 3).

Removing execution trace feedback. Restricting the critic to aggregate latency and energy is the only ablation that degrades hypervolume substantially on every benchmark (0.066–0.111) and yields the largest mean drop (0.082); the remaining components are workload-dependent—the critic matters most on HRNet (−0.114) and the workload DAG on Speech-based VQA (−0.106). At iso-latency,

Table 2: Pareto hypervolume and search time on Intel Meteor Lake. N_{eval} : hardware evaluations; $\widehat{HV}$: normalized hypervolume; t : wall-clock time. Parentheses: improvement over the strongest baseline.

Benchmark	Method	$N_{eval} (\downarrow)$	$\widehat{HV} (\uparrow)$	t (hrs) ($\downarrow$)
Depth Anything V2	NSGA-II	580	0.362	12.0
	Bayesian Opt.	579	0.395	12.0
	Random Search	388	0.063	12.0
	TraceDSE	104 (5.6$\times$)	0.489 (+24%)	3.3 (3.6$\times$)
ImageBind	NSGA-II	671	0.606	12.0
	Bayesian Opt.	687	0.450	12.0
	Random Search	593	0.018	12.0
	TraceDSE	75 (8.9$\times$)	0.758 (+25%)	2.1 (5.7$\times$)
HRNet	NSGA-II	723	0.966	12.0
	Bayesian Opt.	587	0.879	12.0
	Random Search	594	0.033	12.0
	TraceDSE	94 (7.7$\times$)	0.977 (+1%)	2.5 (4.8$\times$)
Speech-based VQA	NSGA-II	242	0.651	12.0
	Bayesian Opt.	215	0.104	12.0
	Random Search	152	0.096	12.0
	TraceDSE	27 (9.0$\times$)	0.839 (+29%)	2.3 (5.2$\times$)

energy increases by up to 73% on ImageBind. On two of four benchmarks, the full system achieves low-latency Pareto-front extremes that the ablated system cannot reach—12% lower on ImageBind and 15% lower on DAV2—as the critic loses visibility into per-node performance bottlenecks and can no longer recommend targeted remappings to reduce critical-path latency.

Removing the critic. Passing the raw execution trace directly to the proposer, bypassing the critic and its programmatic tools, degrades hypervolume on all four benchmarks. As raw traces accumulate across iterations, the proposer’s input context grows from approximately 10K to over 250K tokens, degrading the quality of the LLM’s responses. For Speech-based VQA, the proposer produces increasingly redundant proposals, resulting in lower hypervolume than even the aggregate-only variant. These results show that the

Table 3: Ablation study across four benchmarks. All variants use Gemini 3.1 Pro Preview. Δ is the difference in $\overline{HV}$ from the full system.

Benchmark	Variant	$\overline{HV}$	Δ
Depth Anything V2	Full system	0.489	—
	No trace feedback	0.414	−0.074
	No critic	0.467	−0.022
	No search state	0.448	−0.041
	No workload DAG	0.484	−0.005
ImageBind	Full system	0.758	—
	No trace feedback	0.647	−0.111
	No critic	0.749	−0.009
	No search state	0.737	−0.021
	No workload DAG	0.647	−0.111
HRNet	Full system	0.977	—
	No trace feedback	0.901	−0.076
	No critic	0.863	−0.114
	No search state	0.937	−0.040
	No workload DAG	0.976	−0.001
Speech-based VQA	Full system	0.839	—
	No trace feedback	0.773	−0.066
	No critic	0.755	−0.084
	No search state	0.837	−0.002
	No workload DAG	0.733	−0.106

proposer cannot effectively reason over raw trace data and that the critic’s structured analysis is necessary to convert raw traces into feedback the proposer can act on.

Removing the search state. Removing the Pareto front and exploration summary reduces hypervolume on HRNet and DAV2 but has limited effect on ImageBind and Speech-based VQA. On ImageBind, the ablated system’s Pareto front is narrower, failing to reach the low-energy region the full system discovers (12% higher energy at the energy-optimal extreme). The contrast reflects workload structure: HRNet’s repetitive fork-join topology implies the same nodes dominate the critical path across iterations, so without the exploration summary the proposer repeatedly refines the same region; on ImageBind and Speech-based VQA, the bottleneck naturally shifts between independent encoder chains across iterations, exposing different design space regions without explicit tracking.

Removing the workload DAG from the proposer. Replacing the workload DAG in the proposer’s prompt with unordered node names that do not reveal dependencies among them reduces hypervolume significantly on ImageBind and Speech-based VQA but has no measurable effect on HRNet and DAV2. Unlike the trace ablation, which uniformly degrades the front, removing the DAG causes the system to miss specific Pareto-front extremes while remaining competitive elsewhere. On ImageBind, the ablated system uses 18–49% more energy at iso-latency and the full system extends 8% further into the low-energy region. On Speech-based VQA, the full system extends 4% further into the low-latency region. Reaching these extremes requires the proposer to reason about which nodes can execute concurrently on separate PUs—reasoning that depends on the DAG. DAV2 is mostly sequential, leaving little parallel structure to exploit; HRNet’s topology is repetitive across stages, so the critic’s trace analysis compensates.

Table 4: LLM comparison on ImageBind. All runs use the full TraceDSE flow, differing only in the LLM.

LLM	$\overline{HV}$	Tokens (in/out)	t (hrs)	Cost (\$)
Gemini 3.1 Pro Preview	0.758	737K / 55K	2.1	4.52
Claude Opus 4.7	0.791	913K / 138K	2.6	24.00
GPT-5.5	0.777	422K / 124K	2.2	3.90
Qwen3.6 Max Preview	0.749	214K / 88K	3.9	1.19

We present a detailed case study in Appendix A tracing the proposer–critic loop across 12 iterations on HRNet: the critic identifies a terminal-node bottleneck invisible to aggregate metrics, and the proposer combines this with DAG-level reasoning about concurrent execution to produce a 15.4% latency reduction.

5.3 Robustness to LLM Choice and Stochasticity

To evaluate whether TraceDSE’s performance is sensitive to LLM sampling nondeterminism or the choice of LLM, we vary both independently. Across five independent runs of TraceDSE on ImageBind (Gemini 3.1 Pro Preview, temperature 1.0), hypervolume ranges from 0.744 to 0.764 ($\sigma = 0.009$), with $708K \pm 88K$ input and $57K \pm 14K$ output tokens at $\$4.46 \pm 0.65$ per run, indicating stable performance across runs. Repeating the five-run protocol on the remaining three benchmarks yields run-to-run variation of the same order. Table 4 reports TraceDSE results on ImageBind using four different LLMs. All four achieve hypervolume between 0.749 and 0.791—a spread comparable to the run-to-run variance of a single LLM—despite up to $5.3\times$ differences in cost and $1.9\times$ in wall-clock time. This indicates that TraceDSE’s performance is driven by the proposer–critic architecture and trace-analysis tools rather than by the capability of any individual LLM.

6 Conclusion

Deploying AI inference workloads on heterogeneous edge SoCs requires jointly optimizing workload mapping and PU configuration across a combinatorially large design space. However, conventional DSE methods treat the hardware as a black box, relying solely on aggregate performance metrics without exploiting the rich structured information available in execution traces. To address this, we present TraceDSE, an agentic DSE flow that analyzes execution traces with programmatic tools to convert each hardware evaluation into actionable feedback for guiding the search. On four single-model and multi-model neural network inference workloads deployed on an Intel Meteor Lake edge SoC, TraceDSE consistently identifies higher-quality candidates and exceeds the Pareto hypervolume of NSGA-II by up to 35% and Bayesian optimization by up to 68%, while requiring $\sim 6\text{--}9\times$ fewer hardware evaluations.

Acknowledgments

This material is based upon work supported by the U.S. Department of Energy, Office of Science, Basic Energy Sciences, as part of The Extreme Lithography & Materials Innovation Center (ELMIC), a Microelectronics Science Research Center (MSRC), under Contract No. DE-AC02-06CH11357. We also acknowledge the support for the Microelectronics Seed Model Team effort from the Transformational AI Models Consortium (ModCon) as part of the DOE Genesis Mission.

References

- [1] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. 2011. Algorithms for Hyper-Parameter Optimization. In *Advances in Neural Information Processing Systems*, Vol. 24.
- [2] Behzad Boroujerdian, Ying Jing, Devashree Tripathy, Amit Kumar, Lavanya Subramanian, Luke Yen, Vincent Lee, Vivek Venkatesan, Amit Jindal, Robert Shearer, and Vijay Janapa Reddi. 2023. FARS: An Early-Stage Design Space Exploration Framework to Tame the Domain-Specific System-on-Chip Complexity. *ACM Transactions on Embedded Computing Systems* 22, 2, Article 31 (2023), 35 pages. doi:10.1145/3544016
- [3] Jiasi Chen and Xukan Ran. 2019. Deep Learning With Edge Computing: A Review. *Proc. IEEE* 107, 8 (2019), 1655–1674. doi:10.1109/JPROC.2019.2921977
- [4] Abhijit Das, Enrico Russo, and Maurizio Palesi. 2024. Multi-Objective Hardware-Mapping Co-Optimisation for Multi-DNN Workloads on Chiplet-Based Accelerators. *IEEE Trans. Comput.* 73, 8 (2024), 1883–1898. doi:10.1109/TC.2024.3386067
- [5] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. 2002. A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* 6, 2 (2002), 182–197.
- [6] Zuoming Fu, Alex Manley, and Mohammad Alian. 2025. gem5 Co-Pilot: AI Assistant Agent for Architectural Design Space Exploration. In *3rd Workshop on Computer Architecture Modeling and Simulation (CAMS), co-located with MICRO*. arXiv:2510.19577.
- [7] Rohit Girdhar, Alaeldin El-Nouby, Zhuang Liu, Mannat Singh, Kalyan Vasudev Alwala, Armand Joulin, and Ishan Misra. 2023. ImageBind: One Embedding Space To Bind Them All. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 15180–15190.
- [8] Google DeepMind. 2026. Gemini 3.1 Pro: A Smarter Model for Your Most Complex Tasks. <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/>.
- [9] Intel Corporation. 2024. OpenVINO Toolkit. <https://docs.openvino.ai>.
- [10] Srivatsan Krishnan, Amir Yazdanbakhsh, Shvetank Prakash, Jason Jabbour, Ikechukwu Uchendu, Susobhan Ghosh, Behzad Boroujerdian, Daniel Richins, Devashree Tripathy, Aleksandra Faust, and Vijay Janapa Reddi. 2023. ArchGym: An Open-Source Gymnasium for Machine Learning Assisted Architecture Design. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA '23)*. Association for Computing Machinery. doi:10.1145/3579371.3589049
- [11] Jingyuan Li, Jianrong Zhang, Ye Li, Wenbo Yin, and Lingli Wang. 2025. LEMOE: LLM-Enhanced Multi-Objective Bayesian Optimization for Microarchitecture Exploration. In *Proceedings of the 62nd ACM/IEEE Design Automation Conference (DAC '25)*. 1–7. doi:10.1109/DAC63849.2025.11132704
- [12] Runkai Li, Jia Xiong, and Xi Wang. 2025. iDSE: Navigating Design Space Exploration in High-Level Synthesis Using LLMs. arXiv:2505.22086 [cs.AR]
- [13] Tennison Liu, Nicolás Astorga, Nabeel Seedat, and Mihaela van der Schaar. 2024. Large Language Models to Enhance Bayesian Optimization. In *The Twelfth International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=OOxotBmGol>
- [14] Sujay Pandit, Akanksha Jain, Rami Cohen, Zhijie Deng, Sagar Karandikar, Sagi Perel, Anand Raghunathan, and Parthasarathy Ranganathan. 2025. LLM-Box: An Agentic Framework for Guided Black-Box Optimization in Mapping LLMs onto Specialized Hardware Accelerators. In *Machine Learning for Systems (MLForSys) Workshop at NeurIPS*.
- [15] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning (ICML)*. 8748–8763.
- [16] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. Robust Speech Recognition via Large-Scale Weak Supervision. In *International Conference on Machine Learning (ICML)*. 28492–28518.
- [17] Chirag Sakhuja, Zhan Shi, and Calvin Lin. 2023. Leveraging Domain Information for the Efficient Automated Design of Deep Learning Accelerators. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 287–301. doi:10.1109/HPCA56546.2023.10071095
- [18] Ke Sun, Bin Xiao, Dong Liu, and Jingdong Wang. 2019. Deep High-Resolution Representation Learning for Human Pose Estimation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 5693–5703.
- [19] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. 2002. Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing. *IEEE Transactions on Parallel and Distributed Systems* 13, 3 (2002), 260–274. doi:10.1109/71.993206
- [20] Hanyu Wang, Xinrui Wu, Zijian Ding, Su Zheng, Chengyue Wang, Neha Prakriya, Tony Nowatzki, Yizhou Sun, and Jason Cong. 2025. LLM-DSE: Searching Accelerator Parameters with LLM Agents. arXiv:2505.12188 [cs.AR]
- [21] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. 2024. Qwen2.5 Technical Report. arXiv:2412.15115 [cs.CL]
- [22] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. 2024. Depth Anything V2. In *Advances in Neural Information Processing Systems*.
- [23] Eckart Zitzler and Lothar Thiele. 1999. Multiobjective Evolutionary Algorithms: A Comparative Case Study and the Strength Pareto Approach. *IEEE Transactions on Evolutionary Computation* 3, 4 (1999), 257–271.

A Appendix: Case Study — Proposer–Critic Reasoning on HRNet

We trace the proposer–critic loop across 12 iterations on HRNet to show how execution-trace feedback drives iterative improvement. HRNet’s 98-node DAG ends in the four terminal nodes `hrnet_94`–`hrnet_97` (Figure 4(a)): mutually independent sinks that carry the largest per-node latencies in the workload (41–70 ms each). Their placement therefore largely determines end-to-end latency. Italicized text is quoted verbatim from the run.

Iterations 0–1. Before the loop, one-time profiling measured each node’s latency and energy on every PU at maximum resources; these priors warm-start the proposer and give the critic a contention-free reference for interpreting traces. In Iteration 1 the proposer first distributed the 98 nodes across all three PUs and reached 283 ms—28% better than the all-GPU seed—placing `hrnet_94` on the CPU, which the priors rank fastest for it in isolation (40.2 ms). The trace told a different story: `hrnet_94` took 64.2 ms because the configuration enabled only half the CPU cores (“... *CPU has only 8 of 16 cores active, causing hrnet_94 to run slower than expected*”), so the critic recommended the GPU (a predicted 1.54× speedup). The proposer began testing that placement immediately—once in Iteration 2 and three times in Iteration 3—but spent Iterations 2–5 mainly on the 94 non-terminal nodes, and the terminal assignment did not settle; the front grew to span 281–352 ms / 8.2–9.8 J. The GPU placement only became decisive in Iteration 6, once it was paired with the balanced split.

Iteration 6. The proposer then assigned two terminal nodes to the GPU and two to the NPU, reasoning that spreading independent work across PUs avoids serializing it; the candidate reached 287 ms / 8.3 J, improving both objectives (a new Pareto point). From the trace the critic identified the next bottleneck: `hrnet_96` at 57 ms on the NPU versus a predicted 46.6 ms on the GPU (1.22×).

Iterations 7–8. In Iteration 7 the proposer moved `hrnet_94` back to the CPU with all 16 cores active, probing whether CPU placement could reduce GPU energy; every candidate was dominated (307–353 ms). The critic’s trace analysis explained why: contention from other CPU-mapped nodes inflated `hrnet_94` to 63.1 ms, far above its 40.2 ms contention-free prior (“*Terminal nodes are extremely sensitive ... CPU is too slow even with all cores active*”), so it recommended moving the node off the CPU and, since the CPU would no longer run terminal nodes, halving its allocation to eight cores.

Of the six Iteration-8 candidates—all adopting the configuration change—four followed the critic and placed `hrnet_94` on the NPU, while two placed it on the GPU, the proposer reasoning from the DAG that the four terminal nodes can run concurrently if spread across PUs (“*Extreme Latency Push: Perfectly splits the 4 terminal heads across the accelerators ... for balanced tail execution*”). Call the two single-node-difference candidates X (`hrnet_94`→NPU) and Y (`hrnet_94`→GPU); both use identical PU configurations. As Figure 4(b) shows, X serializes three terminal nodes on the NPU (351.6 ms, dominated) and loses 54 ms to Y’s concurrent split

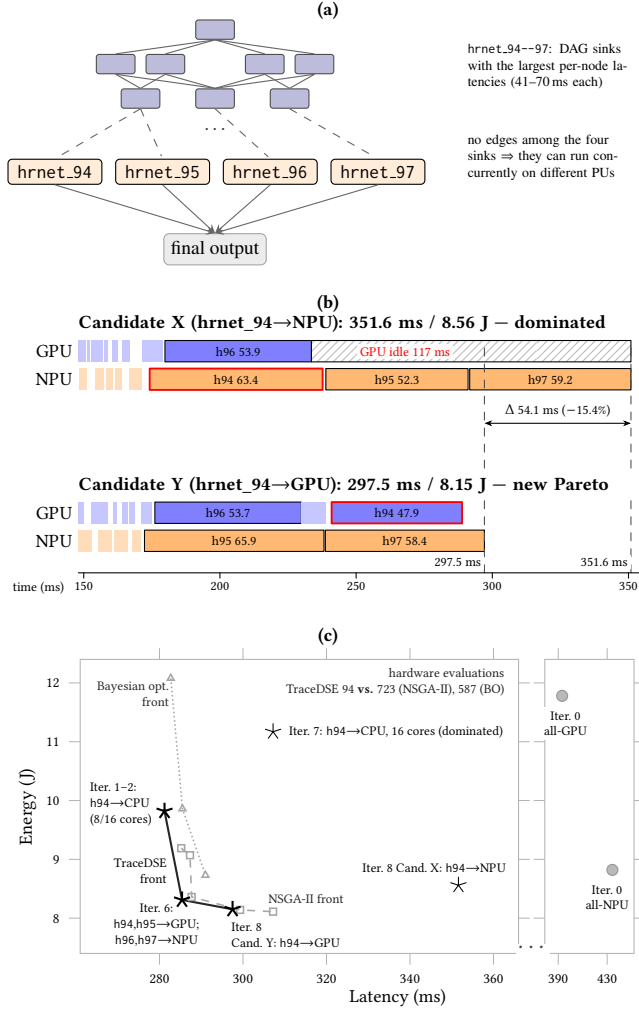


Figure 4: Case study on HRNet. (a) Terminal fragment of the workload DAG: the four sink nodes are mutually independent and can run concurrently on different PUs. (b) Execution traces of the two Iteration-8 candidates differing in a single node: Candidate X (hrnet_94 → NPU) serializes three terminal nodes and idles the GPU for 117 ms (dominated); Candidate Y (hrnet_94 → GPU) runs both accelerators concurrently, 54.1 ms (15.4%) faster (new Pareto point). (c) Search trajectory from the Iteration-0 seeds—all-GPU and all-NPU, i.e. every node mapped to a single PU—to TraceDSE's final Pareto front (axis break marked by dots). Each labeled point gives the iteration and the mapping decision that defines it; dashed and dotted lines are the NSGA-II and Bayesian-optimization final fronts after their full 12-hour budgets. Throughout, h94 abbreviates hrnet_94.

(297.5 ms, new Pareto)—far more than the 15.5 ms per-node difference between the two placements (47.9 vs. 63.4 ms).

Iterations 9–12. The critic generalized: “An exactly balanced 2/2 split between GPU and NPU is mandatory for sub-300 ms latency”—two terminal nodes on each accelerator—and predicted only 5–10 ms

variance across permutations. Iterations 9–10 confirmed this (291–301 ms); energy-focused configurations in Iterations 11–12 added no Pareto points, and the search terminated since five iterations resulted in hypervolume improvement below the 0.5% threshold.

Summary. The agents contribute complementary evidence: the critic's trace analysis corrects priors that contention makes misleading (40.2 ms in isolation versus 63–64 ms in situ) and turns each evaluation into a targeted recommendation, while the proposer adds DAG-level reasoning about concurrency that traces alone do not provide. Together they drive the search to a balanced GPU–NPU terminal split with a reduced CPU allocation, producing the 15.4% latency reduction cited in Section 5.2.